



Algoritma Proses Penjdawalan

Asep Rizal Nurjaman

Algoritma Penjadwalan Proses

- Priority Scheduling
- Round Robin

Priority Scheduling

- Merupakan algoritma yang mendahulukan proses yang memiliki prioritas tertinggi
- Prioritas proses ditentukan berdasar:
 - Time limit
 - Memory requirement
 - File access
 - Perbandingan antara burst proses dengan CPU
 - Tingkat kepentingan proses

PS (2)

- Priority scheduling dapat dijalankan secara preemptive dan non-preemptive
 - Preemptive → jika ada proses yang baru datang memiliki prioritas lebih tinggi dari proses yang sedang berjalan, maka proses yang sedang berjalan tsb dihentikan, lalu CPU dialihkan untuk proses yang baru datang tersebut
 - Non preemptive → proses yang baru datang tidak dapat mengganggu proses yang sedang berjalan, tapi hanya diletakkan di depan queue

PS (3)

- Kelemahan PS adalah terjadinya infinite blocking (**starvation**), yaitu suatu proses dengan prioritas yang rendah memiliki kemungkinan tidak pernah dieksekusi jika terdapat proses lain yang memiliki prioritas lebih tinggi
- Solusi dari starvation adalah **aging**, yaitu meningkatkan prioritas dari setiap proses yang menunggu dalam queue secara bertahap

PS (4)

- Contoh : setiap 10 menit, prioritas dari masing-masing proses yang menunggu dalam queue dinaikkan 1 tingkat.
- Maka proses yang memiliki prioritas 127, setidaknya dalam 21 jam 20 menit, proses tsb akan memiliki prioritas 0, yaitu prioritas yang tertinggi

Contoh 2 - PS

- Diketahui 5 proses dengan urutan proses

Proses	Burst Time (ms)	Prioritas
P1	7	3
P2	2	4
P3	3	3
P4	2	1
P5	10	2



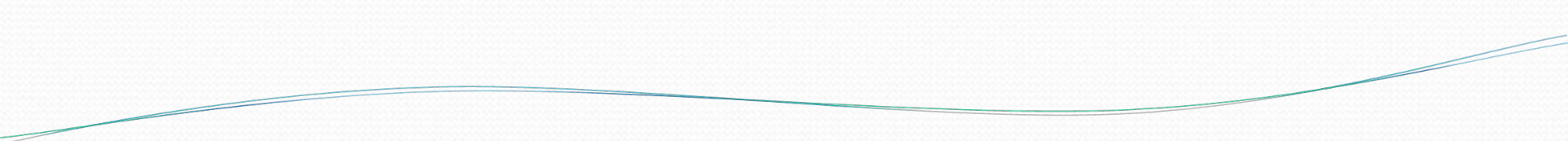
Proses	Waiting Time (ms)
P1	12
P2	22
P3	19
P4	0
P5	2

ΔT_{avg}

$$\begin{aligned}
 AWT &= \frac{12+22+19+0+2}{5} \\
 &= \frac{55}{5} \\
 &= 11 \text{ ms}
 \end{aligned}$$

ROUND ROBIN

- Algoritma ini menggilir proses yang ada di antrian. Proses akan mendapat jatah sebesar *time quantum*. Jika *time quantum*-nya habis atau proses sudah selesai, CPU akan dialokasikan ke proses berikutnya
- Proses ini cukup adil, karena tidak ada proses yang diprioritaskan
- Semua proses mendapat jatah waktu yang sama dari CPU yaitu $1/n$, dan tidak akan menunggu lebih lama dari $(n-1)q$; dimana q adalah lama 1 quantum

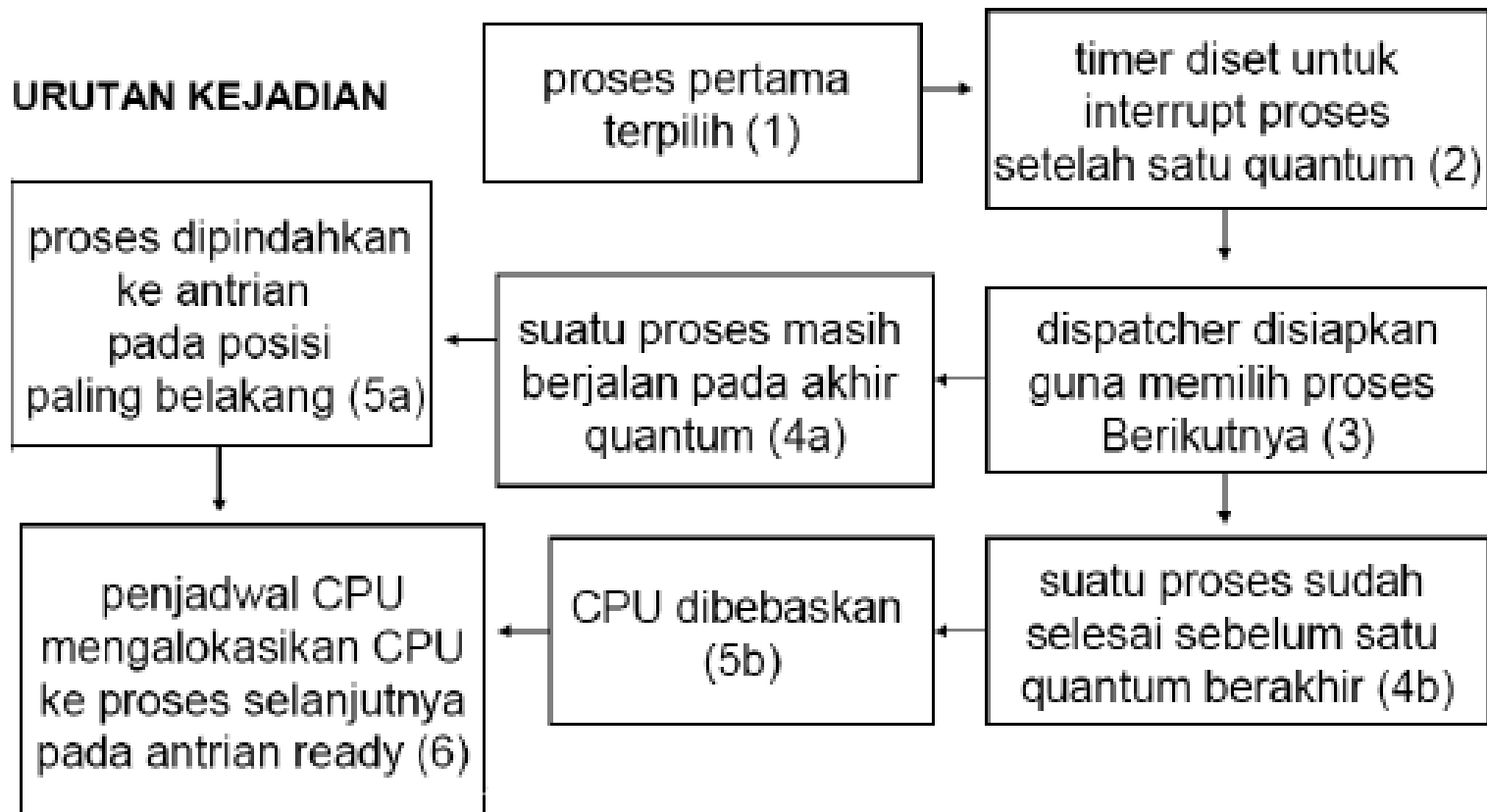
- 
- Algoritma RR sepenuhnya bergantung besarnya time quantum (TQ).
 - Jika TQ terlalu besar, algoritma ini akan sama saja dengan algoritma FCFS
 - Jika TQ terlalu kecil, akan semakin banyak peralihan proses sehingga banyak waktu yang terbuang

Permasalahan algoritma RR

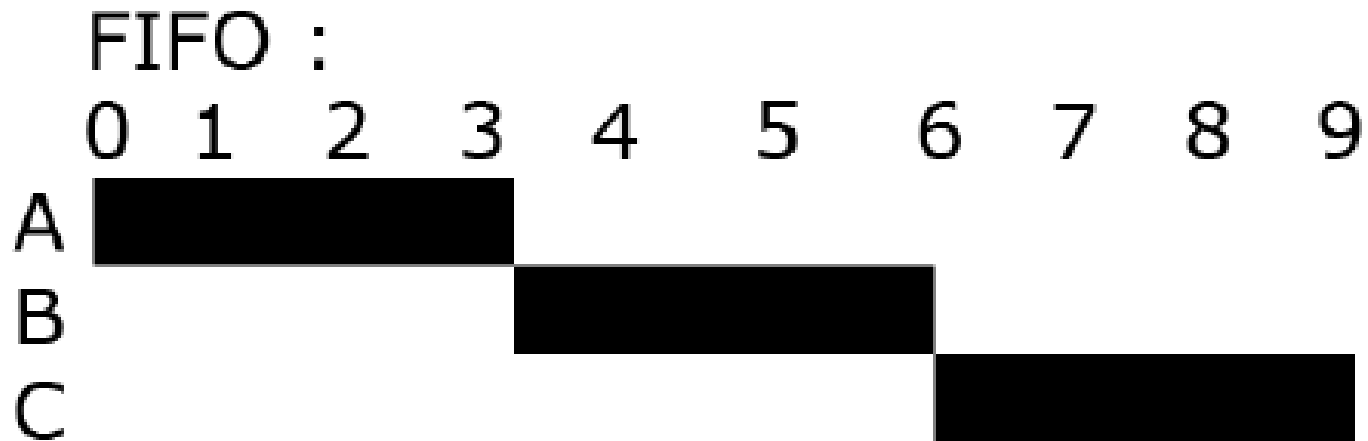
- Permasalahan utamanya adalah menentukan besarnya TQ. Jika TQ yang ditentukan terlalu kecil, maka sebagian besar proses tidak akan selesai dalam 1 quantum.
- Akibatnya akan terjadi banyak switch, padahal CPU memerlukan waktu untuk beralih dari satu proses ke proses yang lain (= context switches time)
- Sebaliknya, jika TQ yang ditentukan terlalu besar, algoritma RR akan berjalan seperti FCFS
- TQ ideal adalah jika 80% dari total proses memiliki CPU burst time yang lebih kecil dari 1 TQ

Urutan Kejadian RR

URUTAN KEJADIAN



Penggunaan TQ



Contoh sederhana

Proses	Durasi	Urutan	Kedatangan
P1	3	1	0
P2	4	2	0
P3	3	3	0

Asumsi: kuantum waktu=1 unit; P1, P2, & P3 tidak pernah diblokir



Contoh 2

- Diketahui 3 proses sbb:

Proses	Burst Time (ms)
P1	23
P2	6
P3	6

- $TQ = 3$

- Gantt chart

P1	P2	P3	P1	P2	P3	P1	P1	P1	P1	P1	P1	
0	3	6	9	12	15	18	21	24	27	30	33	35

- Burst Time

Proses	Burst Time (ms)
P1	$0 + (9 - 3) + (18 - 12) = 12$
P2	$3 + (12 - 6) = 9$
P3	$6 + (15 - 9) = 12$

$$\begin{array}{r}
 \text{AWT} = \frac{12 + 9 + 12}{3} \\
 = 33 \\
 \frac{33}{3} \\
 = 11 \text{ ms}
 \end{array}$$

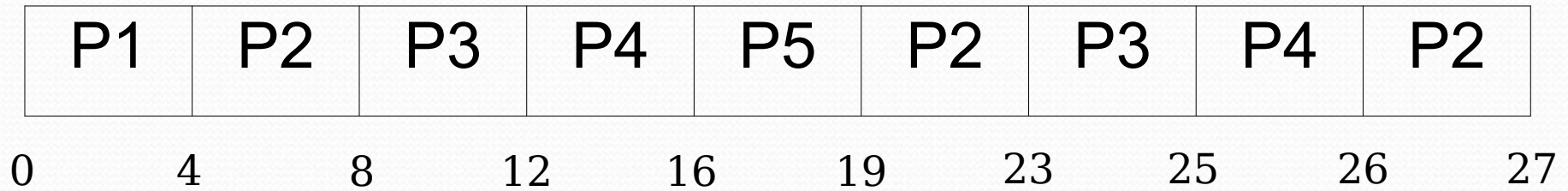
Contoh 3 - RR

- Sebuah CPU dengan quantum 4 mendapat beberapa proses yang kedatangannya sebagai berikut:

Proses	Burst Time
P1	4
P2	9
P3	6
P4	5
P5	3

► Burst time → waktu proses

Gantt Chart



AWT (average waiting time)

- Waktu tunggu untuk tiap-tiap proses :

Proses	Waiting Time
P1	0
P2	$4 + (19 - 8) + (26 - 23) = 18$
P3	$8 + (23 - 12) = 19$
P4	$12 + (25 - 16) = 21$
P5	16

- AWT yang terjadi adalah:
 - $(0 + 18 + 19 + 21 + 16) / 5 = 74 / 5 = 14,8$

ATR (average turn around)

Proses	Saat Tiba	Lama Proses	Saat Mulai	Saat Selesai	Turn Around Time
P1	0	4	0	4	4
P2	0	9	4	27	27
P3	0	6	8	25	25
P4	0	5	12	26	26
P5	0	3	16	19	19
				Jumlah Rata-rata	101 20,2

Round Robin dengan Waktu Tiba berbeda

Nama Proses	Saat Tiba	Lama Proses
A	0	5
B	1	3
C	5	7
D	6	1
E	7	6
	Jml	22

Quantum =
2

Gantt Chart

A	A	B	B	A	A	B	C	C	D	A	
---	---	---	---	---	---	---	---	---	---	---	--

0 1 2 3 4 5 6 7 8 9 10

E	E	C	C	E	E	C	C	E	E	C	
---	---	---	---	---	---	---	---	---	---	---	--

1 1 1 1 1 1 1 1 19 20 21 2
1 2 3 4 5 6 7 8 2

AWT

Nama Proses	Saat Tiba	Lama Proses	Waiting Time
A	0	5	$0+(4-2)+(10-6)=6$
B	1	3	$(2-1)+(6-4)=3$
C	5	7	$(7-5)+(13-9)+(17-15)+(21-19) = 10$
D	6	1	$(9-6)=3$
E	7	6	$(11-7)+(15-13)+(19-17)=8$

- $AWT = (6+3+10+3+8)/5 = 30/5 = 6$

Turn Around

Nama Proses	Saat Tiba	Lama Proses	Saat Mulai	Saat Selesai	Lama Proses
A	0	5	0	11	11
B	1	3	2	7	6
C	5	7	7	22	17
D	6	1	9	10	4
E	7	6	11	21	14
				Jumlah Rata-rata	52 10,4

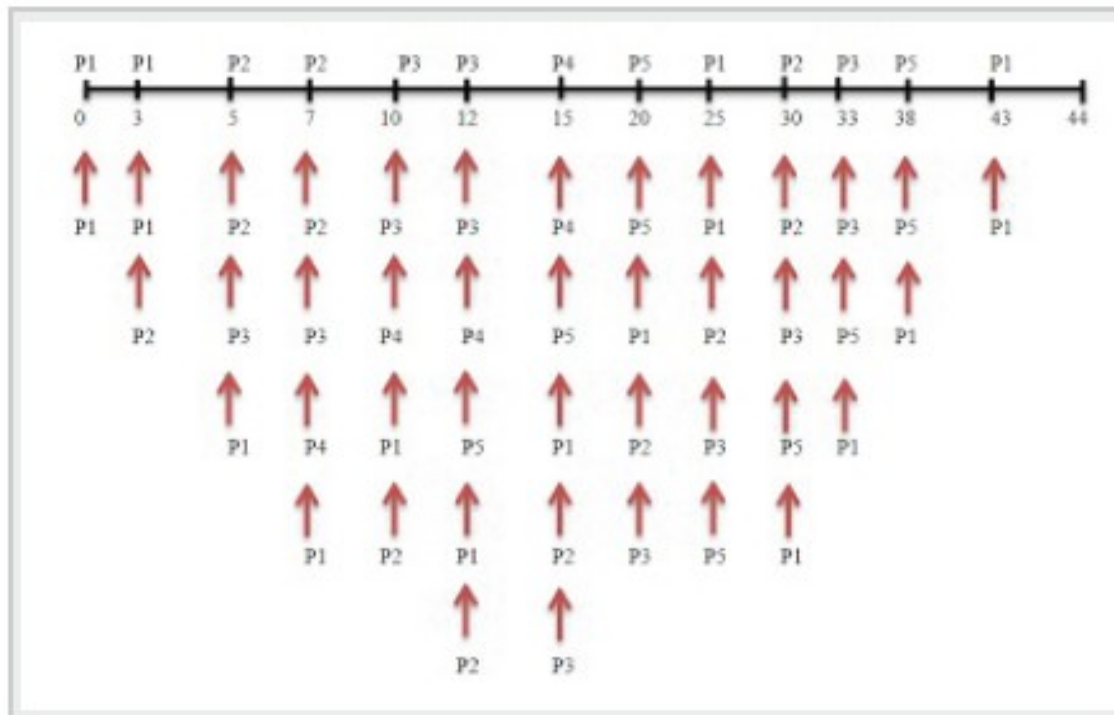
Contoh 4

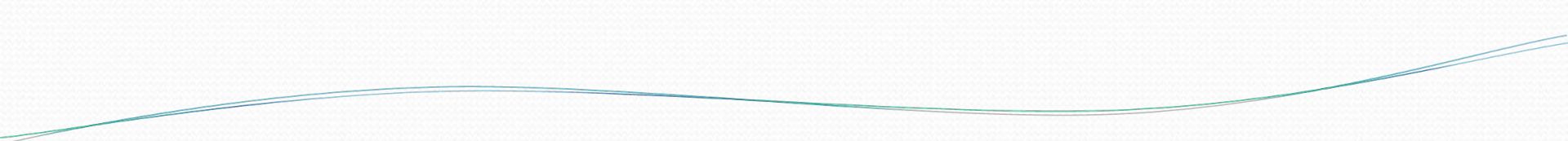
- Untuk memahami dari cara kerja algoritma penjadwalan Round Robin ini, mari kita kerjakan soal berikut

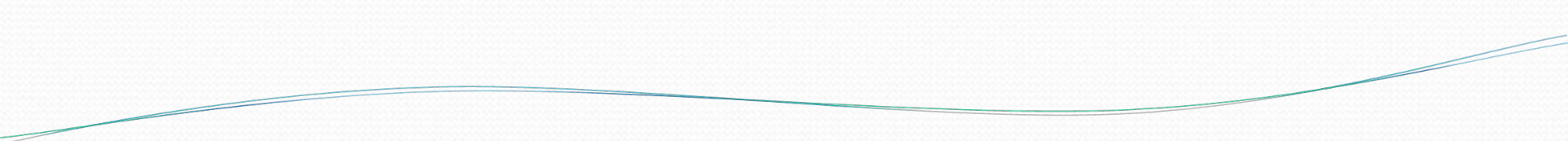
Process ID	Priority	Arrival Times(ms)	Burst Times(ms)
P1	3	0	11
P2	2	3	8
P3	3	5	10
P4	1	7	5
P5	2	12	10

- Penyelesaian :

- Seperti halnya algoritma penjadwalan sebelumnya, langkah pertama untuk mencari AWT dengan Algoritma penjadwalan Round Robin dilakukan dengan membuat Gar.



- 
- Dari Gantt Chart terlihat bahwa setiap proses dikerjakan menurut waktu yaitu setiap proses di proses sebesar 5 langkah.
 - Awalnya P1 akan di kerjakan sebanyak 5 langkah, kemudian, P2 sebanyak 5 langkah, dan begitupun selanjutnya hingga P5.
 - Proses yang sudah di proses menurut porsi waktu yang diberikan akan kembali menunggu dan berada paling belakang dari antrian proses yang ada.

- 
- Contohnya P1 dikerjakan di awal, kemudian ada P2, P3, P4, dan P5 yang mengantri di belakangnya.
 - Jika P1 selesai di proses menurut porsi waktunya maka P1 akan di pindahkan ke belakang, sehingga urutannya menjadi P2, P3, P4, P4, P1. begitupun seterusnya.

● Waiting Time

Proses	WT(ms)
P1	$(0-0)+(25-5)+(43-30)=33$
P2	$(5-3)+(30-10)=22$
P3	$(10-5)+(33-15)=23$
P4	$15-7=8$
P5	$(20-12)+(38-25)=21$
Total	107

● AWT

$$\begin{aligned} \text{AWT} &= \frac{\sum Wt}{P} \\ &= \frac{107}{5} \\ &= 21.4 \text{ ms} \end{aligned}$$